SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Debugging or Optimization task

Code of Conduct:

*I **Subash G [192572150]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

QUESTIONS

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(20 Marks)(BL4)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(20 Marks)(BL4)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(20 Marks)(BL4)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(20 Marks)(BL4)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.
Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(20 Marks)(BL4)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

TOTAL MARKS AWARDED **FACULTY SIGNATURE**

SOLUTIONS

Debugging and Optimisation of Process, Memory and Scheduling Problems

Q1. Deadlock Debugging (Code Trace) – Solution

Problem Identification

Two processes share two resources, R1 and R2. P1 locks R1 and then R2. P2 locks R2 and then R1. Both processes wait 100 ms between their first and second lock. The two processes therefore ask for the same two locks in the **opposite order**. This is the classic situation that leads to a deadlock.

Debugging: Step-by-Step Code Trace

I traced the two processes on a time line. Because of the 100 ms wait, both processes get their first lock before either one asks for the second lock.

Time	P1	P2	System state
0 ms	lock(R1) succeeds	lock(R2) succeeds	P1 holds R1, P2 holds R2
0 to 100 ms	wait(100 ms)	wait(100 ms)	Both still hold their first lock
100 ms	lock(R2) blocks, R2 is held by P2	lock(R1) blocks, R1 is held by P1	Each process waits for the other
After 100 ms	Blocked for ever	Blocked for ever	Deadlock: no progress

Table 1.1: Trace of the original code

The four necessary conditions for a deadlock (Coffman conditions) were then checked one by one.

Condition	How it appears in this code	Present?
Mutual exclusion	A lock can be owned by only one process at a time	Yes
Hold and wait	P1 holds R1 while asking for R2; P2 holds R2 while asking for R1	Yes

No pre-emption	A lock cannot be taken away; only its owner can unlock it	Yes
Circular wait	P1 waits for R2 (held by P2) and P2 waits for R1 (held by P1)	Yes

Table 1.2: Coffman conditions

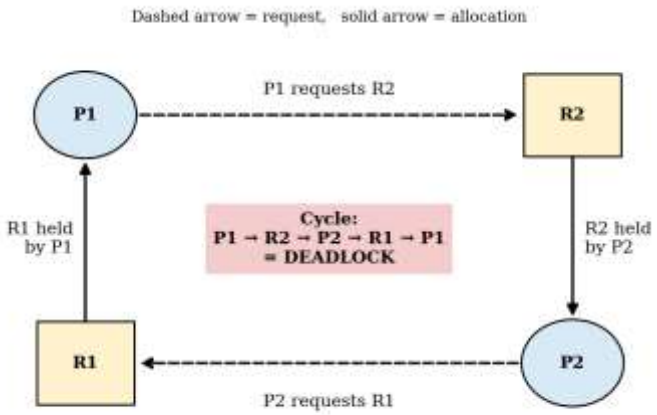


Figure 1.1: Resource allocation graph of the original code. The cycle shows the deadlock.

Exact Condition That Causes the Deadlock

The deadlock happens when **P1 holds R1 and waits for R2 at the same time as P2 holds R2 and waits for R1**. In the resource graph this is the cycle P1 → R2 → P2 → R1 → P1. The root cause is that the two processes lock the resources in different orders. The 100 ms wait does not create the problem; it only makes the bad timing certain. Without the wait, the deadlock would be a rare timing bug (a race) that appears only sometimes and is very hard to reproduce.

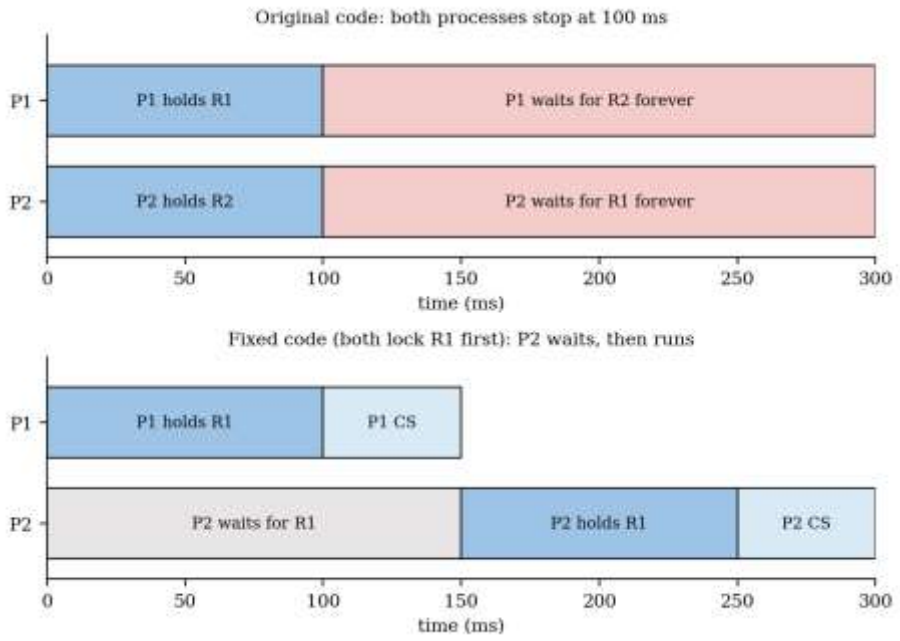


Figure 1.2: Time line before the fix (deadlock) and after the fix (no deadlock)

Optimisation: Remove the Deadlock Without Changing the Process Logic

The fix is a **global lock ordering**. Every process must acquire the resources in the same fixed order: R1 first, then R2. Only the order of the two lock statements in P2 is swapped. The critical section, the waits and the unlocks stay the same, so the logic of both processes is unchanged.

```
Process P1:              Process P2 (corrected):
  lock(R1)                 lock(R1)   // was lock(R2)
  wait(100 ms)             wait(100 ms)
  lock(R2)                 lock(R2)   // was lock(R1)
  critical section         critical section
  unlock(R2)               unlock(R2) // reverse order
  unlock(R1)               unlock(R1)
```

Why it works: if every process takes the locks in increasing order, a process that holds R2 never waits for R1. A wait can only point from a lower resource to a higher one, so a cycle can never be formed and the circular-wait condition is broken. Breaking any one of the four conditions is enough to prevent a deadlock.

Global lock ordering	Circular wait	No extra cost and no change in logic;

(chosen)		needs every process to follow the rule
Take both locks together or none	Hold and wait	Safe, but less concurrency
try-lock with time-out and retry	Hold and wait	Works, but can cause repeated retries (livelock)
Detect the cycle and kill a process	Recovery	Wastes work and adds run-time cost

Table 1.3: Ways to remove the deadlock

Verification

I wrote a small program with two threads and two locks. A time-out was added only so that the demonstration could finish and report what happened. The result is shown below.

```
Original code (P1: R1 then R2, P2: R2 then R1)
  P1: stuck, holds R1, waits for R2 -> DEADLOCK
  P2: stuck, holds R2, waits for R1 -> DEADLOCK
Fixed code (both: R1 then R2)
  P1: entered critical section with R1 and R2
  P2: entered critical section with R1 and R2
```

In the demonstration the original code deadlocked in every run, while in the corrected code both processes entered the critical section one after the other, with no extra run-time cost.

Summary

The deadlock is caused by a circular wait created by opposite lock orders. Making both processes lock R1 before R2 removes the cycle, keeps the process logic unchanged, and lets both processes finish.

Q2. CPU Utilisation Problem – Solution

Problem Identification

The CPU is busy only 30% of the time although processes are in the system. The scheduler log shows the repeated pattern **ready → waiting → ready → waiting → ready**. A process that is

running should normally stay running until its time slice ends. Here the processes leave the CPU very quickly to go to the waiting state, which means they ask for I/O after a very short CPU burst. These are **I/O-bound** processes.

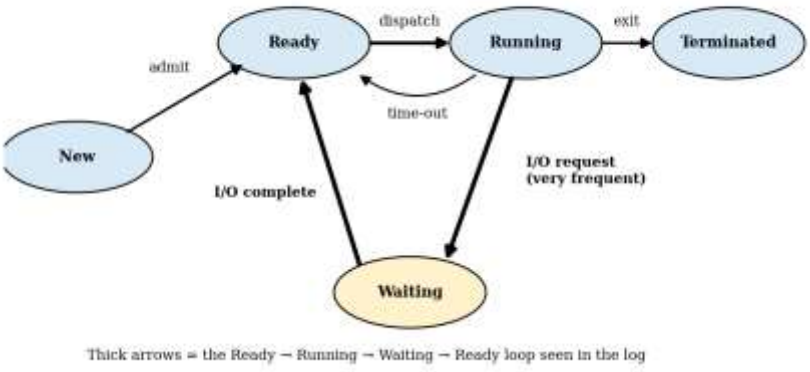


Figure 2.1: Process state diagram. The thick arrows show the loop found in the log.

Debugging

1. **Read the log pattern.** Almost every ready period ends in the waiting state, and almost every waiting period ends in ready. Processes do very little computing between I/O requests.
2. **Estimate the burst and the wait.** For the calculation I use a CPU burst of 3 ms followed by an I/O wait of 17 ms. These values are an assumption chosen to match the 30% in the question, because the log gives only the order of the states.
3. **Use the utilisation formula.** With n processes that overlap their I/O, CPU utilisation = n × burst / (burst + I/O wait), up to 100%. For two processes this is 2 × 3 / 20 = 30%, which matches the observation.
4. **Rule out other causes.** The other possible causes were checked and are listed in Table 2.1.

Possible cause	Evidence in the log	Verdict
Time quantum too long	Processes leave the CPU by themselves, not by time-out	Not the cause
Scheduler or context-switch overhead	Burst is short but the CPU is idle, not busy switching	Not the cause

One CPU-bound process hogging the CPU	No long running periods are seen	Not the cause
Too few processes to overlap I/O	All processes are waiting at the same time, so the CPU idles	Root cause

Table 2.1: Elimination of possible causes

Root Cause

Each process computes for only about 3 ms and then waits about 17 ms for I/O. With only two processes in memory, both are often in the waiting state together, so the CPU has nothing to run and sits idle. The ready queue empties quickly, so the words "processes are ready" are only true for a very short moment. The low utilisation is caused by too little overlap between the CPU and the I/O, not by the CPU being slow.

Optimisation

- **Increase the degree of multiprogramming.** To keep the CPU busy, I need at least (3 + 17) / 3 = 6.7, so 7 processes must be available. When one process waits for I/O, another one is ready to run.
- **Give I/O-bound processes higher priority (multilevel feedback queue).** A process with a short burst runs first, starts its I/O early, and the I/O device works in parallel with the CPU.
- **Reduce the waiting time itself.** Use asynchronous I/O, buffering and caching so that processes wait less for the device.

Limit: the number of processes cannot be increased without limit, because too many processes make the memory too small and cause thrashing (see Q3). Seven to eight processes are enough here.

Verification

I simulated a single CPU with a first-come first-served ready queue, a 3 ms burst and a 17 ms wait for each process. The simulation ran for 1,000 ms for every value of n.

n	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**

n	1	2	3	4	5	6	7	8
CPU utilisation	15.9%	31.8%	47.7%	63.6%	79.2%	94.8%	100.0%	100.0%

Table 2.2: Simulated CPU utilisation for n processes in memory

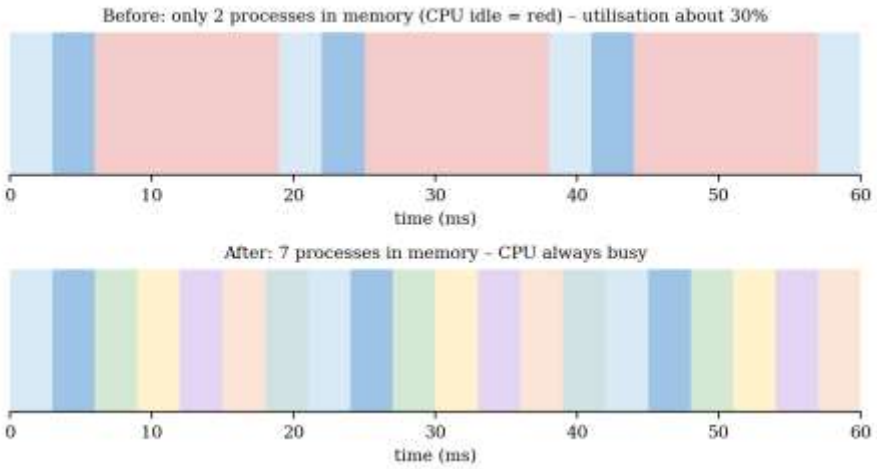


Figure 2.3: CPU time line before (2 processes) and after (7 processes)

With 2 processes the simulated utilisation is 31.8%, which matches the 30% in the question. With 7 processes it reaches 100%. The usual statistical formula 1 − p^n (with p = 0.85) gives a smoother curve with the same trend.

Summary

The CPU is idle because I/O-bound processes spend most of their time waiting and too few processes are available to overlap their I/O. Raising the number of processes to 7 or 8, and scheduling short-burst processes first, increases utilisation from about 30% to nearly 100% in the simulation.

Q3. Memory Bottleneck Debugging – Solution

Problem Identification

A multi-threaded program shows many page faults and runs slowly. The first step is to find out how much memory the program really needs, and compare it with the 2 GB of RAM.

Region	Size	Pages (4 KB each)

Code	5 MB	1,280
Heap	150 MB	38,400
Stacks (25 threads × 1 MB)	25 MB	6,400
Total	180 MB	46,080
Physical RAM	2,048 MB	524,288 frames

Table 3.1: Memory used by the program

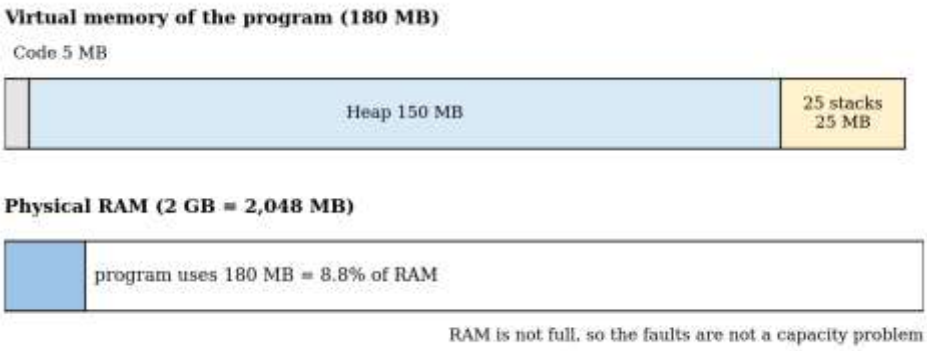


Figure 3.1: Memory used by the program compared with physical RAM

The program needs 180 MB, which is only 8.8% of the 2 GB RAM. The heap is the largest part: 150 / 180 = 83% of the program. So the faults are **not caused by RAM being too small** for this program.

Debugging

Because RAM is large enough, the cause must be the way the memory is used. I would confirm this with pmap -x PID (memory map), ps -o min_flt,maj_flt -p PID (minor faults need no disk, major faults need the disk), vmstat 1 (to see swapping in the si and so columns) and perf stat -e page-faults (faults per second for each thread).

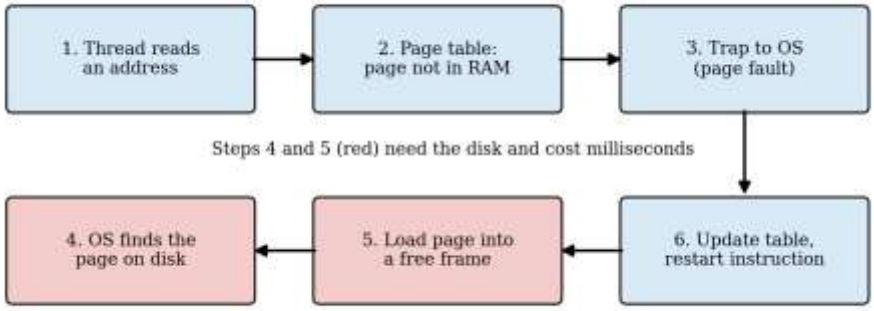


Figure 3.2: Steps of a page fault. The disk steps (red) are the expensive ones.

Memory Issue Identified

- **Large heap with poor locality.** 25 threads work on one 150 MB heap. If they touch it in a scattered way, the set of pages in use at the same time (the working set) is large, and pages are replaced and loaded again and again.
- **Demand paging of a big heap.** Every one of the 38,400 heap pages causes a page fault the first time it is touched, and again if the page is released and reused.
- **Thread overhead.** 25 threads need 25 stacks and 25 separate working sets that compete for the same pages and the same CPU caches.

The cost of faults is large. I use an effective access time model with a memory access of 100 ns and a major fault service time of 8 ms (assumed typical values).

Fault rate p	Effective access time	Slowdown
1 in 10,000,000	100.8 ns	1.0 times
1 in 1,000,000	108 ns	1.1 times
1 in 100,000	180 ns	1.8 times
1 in 10,000	900 ns	9 times
1 in 1,000	8,100 ns	81 times

Table 3.3: Effect of the page-fault rate (EAT = 100 × (1 − p) + 8,000,000 × p)

Only one fault in a million accesses already costs about 8%. One fault in a thousand accesses makes the program about 81 times slower. Therefore the number of faults must be reduced.

Optimisation 1: Improve Locality with Per-Thread Memory Pools

Divide the 150 MB heap into one pool for each thread (150 / 25 = 6 MB each) and let every thread work on its own block of data. Reuse buffers instead of repeated allocation and release. Each thread then keeps a small working set, its pages stay in memory, and threads stop replacing each other's pages.

Optimisation 2: Pre-fault the Heap, Use Huge Pages and a Thread Pool

- **Pre-fault:** touch or map the heap once at start-up (for example with MAP_POPULATE or madvise), so that faults do not happen while the program is working.
- **Huge pages:** with 2 MB pages the heap needs only 150 / 2 = 75 pages instead of 38,400, which is a 99.8% reduction in first-touch faults and in page-table entries.
- **Thread pool:** use about as many threads as CPU cores (for example 8) instead of 25. Stack memory falls from 25 MB to 8 MB, and fewer working sets compete.

Measure	Before	After
Heap pages (first-touch faults)	38,400	75 (2 MB pages)
Stack memory	25 MB (25 threads)	8 MB (8 threads, example)
Heap working set per thread	Whole 150 MB heap	About 6 MB pool
Faults while running	Frequent	Mostly none after pre-fault

Table 3.4: Expected effect of the two optimisations

Summary

The program uses only 8.8% of the RAM, so the page faults come from poor locality, a large shared heap and too many threads, not from a lack of memory. Per-thread pools, pre-faulting with huge pages, and a smaller thread pool reduce the number of faults and the cost of each access.

Q4. Starvation Debugging – Solution

Problem Identification

The scheduler has three queues: Q1 (interactive, round robin, 5 ms), Q2 (system, round robin, 10 ms) and Q3 (batch, FCFS). Process P5 is in Q3 and never gets the CPU. The question is why this happens and how to make the scheduler fair.

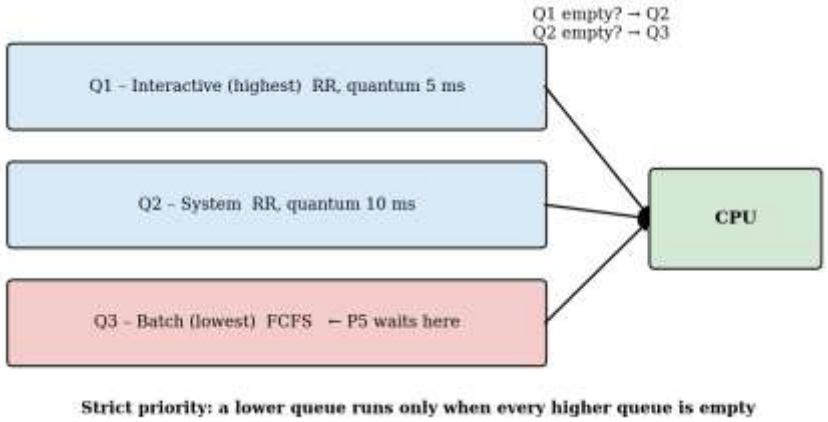


Figure 4.1: The multilevel queue scheduler with strict priority

Debugging

In a plain multilevel queue every queue has absolute priority over the queues below it. The CPU goes to Q2 only when Q1 is empty, and to Q3 only when Q1 and Q2 are both empty. P5 can run only if Q1 and Q2 are empty at the same moment.

I checked whether Q1 ever becomes empty. Assume three interactive processes, each of which needs 5 ms of CPU and then thinks for 8 ms before it needs the CPU again. Their demand is 3 × 5 ms in every 13 ms, that is 15 / 13 = 115% of one CPU. Because the demand is above 100%, there is always at least one interactive process ready, so Q1 never becomes empty and the lower queues never get a turn.

To prove this, I simulated the scheduler for 1,000 ms with these three interactive processes (Q1), one system process that needs 10 ms every 40 ms (Q2), and P5 (Q3) needing 20 ms of CPU, arriving at time 0.

Root Cause

The root cause is **strict priority between queues without any aging**. As long as higher queues are continuously supplied with work, a lower-priority process waits for ever. This is starvation. Q3 being FCFS makes it worse, because a batch job cannot even compete with others inside its own queue.

Optimisation: Aging

Aging slowly increases the priority of a process that has waited a long time. In my design, a process that has waited 50 ms without getting the CPU is moved up one queue (Q3 → Q2 → Q1), and its waiting counter is reset. When it has finished its burst, it goes back to its own queue.

```
every 1 ms, for each waiting process p:
    p.wait = p.wait + 1
    if p.wait >= 50 and p.queue > Q1:
        move p to the next higher queue
        p.wait = 0
when p finishes its burst:
    p returns to its original queue
```

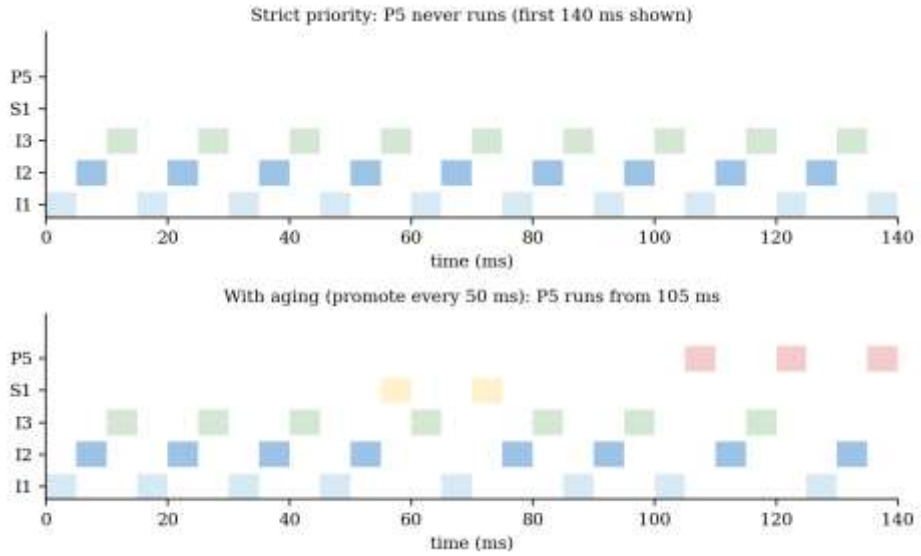


Figure 4.2: Strict priority against aging (CPU time line of the first 140 ms)

P5 waits 50 ms and moves to Q2. It still cannot run because Q1 is always busy, so after another 50 ms it moves to Q1 and is scheduled in round robin with the interactive processes. It first runs at 105 ms and finishes at 155 ms.

Alternatives Considered

Two other fairness methods were also tested. **Time slicing between the queues** (70 ms for Q1, 20 ms for Q2 and 10 ms for Q3 in every 100 ms) guarantees each queue a share of the CPU; P5 starts at 90 ms and ends at 200 ms. A **multilevel feedback queue with a periodic boost** lifts all processes to the top queue regularly; it behaves like aging but needs a more complex design. Aging was chosen because it directly removes the cause (waiting for ever) and is simple to implement.

Verification

Scheduler	**P5 first runs at**	**P5 finishes at**	**P5 CPU in 1,000 ms**	**Average response of the other processes**
Strict priority	Never	Never	0 ms	2.04 ms
Aging	105 ms	155 ms	20 ms	6.39 ms
Queue time slices	90 ms	200 ms	20 ms	7.16 ms

Table 4.3: Comparison of the three schedulers (response = time from ready to first run)

The price of fairness is small: the average response time of the interactive and system processes rises from 2.0 ms to 6.4 ms, which is still too short for a user to notice, and P5 now completes its work.

Summary

P5 starves because Q1 never becomes empty and the queues have strict priority. Adding aging with a 50 ms limit lets P5 reach the CPU after 105 ms and finish at 155 ms, with only a small rise in response time for the other processes.

Q5. Context-Switch Overhead Debugging – Solution

Problem Identification

A server performs 20,000 context switches per second. The average CPU burst of a thread is 2 ms, the threads wait for I/O very often, and the preemptive scheduler uses a quantum of 1 ms. The task is to explain why the number of switches is so high and to reduce the overhead.

Debugging: Why Are There So Many Context Switches?

A context switch happens in two ways. A **forced switch** happens when the quantum ends and the scheduler takes the CPU away. A **voluntary switch** happens when the thread blocks for I/O. Both are present here.

- **Quantum smaller than the burst (forced switches).** The burst is 2 ms but the quantum is 1 ms. Every burst is cut into two slices, so each burst causes one forced switch after the first millisecond.
- **Very frequent I/O (voluntary switches).** At the end of each burst, the thread blocks for I/O and the CPU switches to another thread.

So each 2 ms burst needs **2 switches**: one forced and one voluntary. This agrees with the log: 20,000 switches per second means 20,000 / 2 = 10,000 bursts per second, with 10,000 forced and 10,000 voluntary switches. Half of all the switches are caused only because the quantum is smaller than the burst.

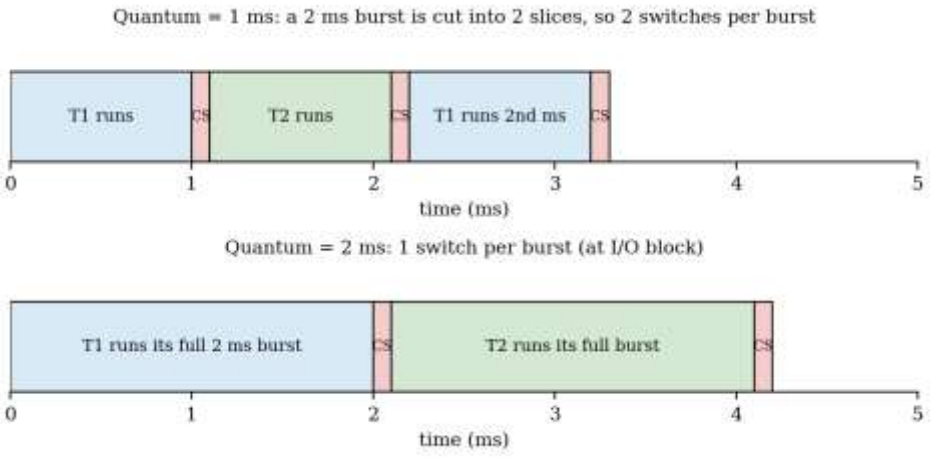


Figure 5.1: One burst with a 1 ms quantum (2 switches) and with a 2 ms quantum (1 switch)

Cost of the Overhead

A context switch costs CPU time for saving and loading registers, entering the kernel, and, most importantly, for refilling the CPU cache and TLB after the new thread starts. I assume a total cost c of 10 µs per switch (typical range is 5 to 20 µs). The CPU time lost in each second is switches per second × c, so the original system loses 20,000 × 10 µs = 0.20 s in every second, which is 20% of one CPU core.

The time lost does no useful work for any user request, and the cache refills also slow down the threads that run after each switch.

Optimisation 1: Increase the Quantum to Match the Burst

Set the quantum to about 2 ms (tuned between 2 ms and 4 ms), so that an average burst finishes inside one time slice. The forced switches disappear, and the total falls from 20,000 to 10,000 per second, a 50% reduction. The response time stays short, because a thread never waits more than a few milliseconds for its turn.

Optimisation 2: Use a Worker Pool with Asynchronous I/O

Instead of one blocking thread for each request, use a small pool of worker threads (about the number of cores) and an event loop (for example epoll). A worker never sleeps on I/O; it starts the I/O, picks the next ready request, and handles the result when the I/O completes. Requests that become ready together can be handled in a batch. If a batch handles on average 5 requests per wake-up (a design target), the voluntary switches fall from 10,000 to 10,000 / 5 = 2,000 per second. Pinning each worker to a core (CPU affinity) also keeps its cache warm.

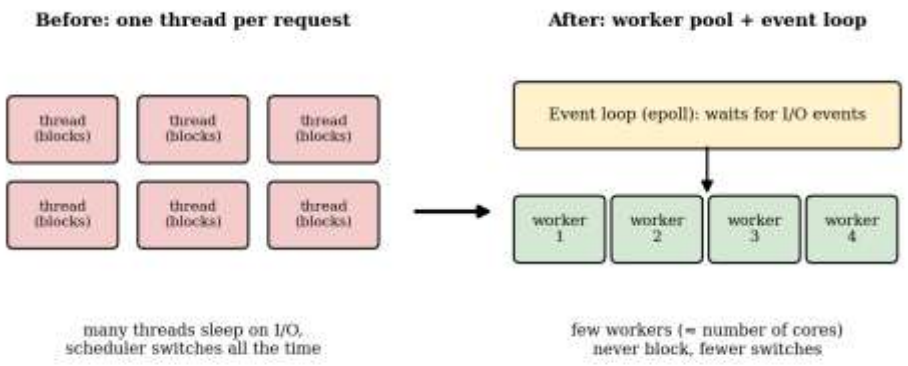


Figure 5.2: Thread-per-request design (before) and worker pool with event loop (after)

Verification

Case	**Switches per second**	**Time lost per second (c = 5 µs)**	**Time lost per second (c = 10 µs)**	**Time lost per second (c = 20 µs)**
Original (quantum 1 ms)	20,000	0.10 s	0.20 s	0.40 s
Optimisation 1 (quantum 2 ms)	10,000	0.05 s	0.10 s	0.20 s
Optimisations 1 and 2	2,000	0.01 s	0.02 s	0.04 s

Table 5.2: Switch rate and CPU time lost

With both optimisations the number of switches falls by 90% and the lost CPU time falls from 0.20 s to 0.02 s in every second (for c = 10 µs). The saved CPU time can serve more requests. **Trade-off:** a longer quantum slightly delays CPU-bound threads that wait for their turn, and the event-driven design is more complex to program. For a server with short bursts, both costs are small compared with the gain.

Summary

The switch rate is high because the 1 ms quantum cuts every 2 ms burst into two slices and because threads block for I/O very often. Raising the quantum to 2 ms and moving to a worker pool with asynchronous I/O reduces the switches from 20,000 to about 2,000 per second.

Overall Summary of All Questions

Question	Root cause found	Optimisation applied	Evidence of improvement
Q1 Deadlock	Opposite lock order gives a circular wait	Global lock order R1 then R2	Both processes finish; no cycle
Q2 CPU utilisation	Too few processes overlap their I/O	7 to 8 processes; favour short bursts	30% to 100% (simulated)
Q3 Memory	Poor locality, large shared heap, 25 threads	Per-thread pools; huge pages and pre-fault; thread pool	Heap pages 38,400 to 75
Q4 Starvation	Strict priority without aging	Aging every 50 ms	P5 finishes at 155 ms instead of never
Q5 Context switches	Quantum 1 ms below the 2 ms burst; frequent I/O	Quantum 2 ms; worker pool with async I/O	20,000 to 2,000 per second

Table 6: Summary of root causes, fixes and results

All five problems were solved in the same way: observe the symptom, find the process-management concept that explains it, prove it with a trace, a calculation or a simulation, apply an optimisation, and check that the result improves. The Python programs used for the traces and the simulations are provided as a separate file with this submission.